Cloudesk

Produ

Everything needed to take the Cloudesk platform live on Amazon Web Services in a scalable, secure shape: the checks that were run on the code, the fixes made in this pass, the target architecture, the step-by-step build-out, the environment, email, payments, customer domains, CI/CD, monitoring, backups, scaling, security hardening and the final go-live checklist.

Prepared 29 October 2026 - for the Cloudesk engineering team. Code version: the working tree at the time of this pass (not yet committed - see section 1).

Contents

- 1. Where things stand
- 2. Target architecture on AWS
- 3. Build-out, step by step
- 4. Environment and secrets
- 5. Email, payments and AI
- 6. Customer domains and TLS
- 7. First release and bootstrap
- 8. CI/CD
- 9. Monitoring, logs and alarms
- 10. Backups and disaster recovery
- 11. Scaling
- 12. Security hardening
- 13. Go-live checklist
- 14. Known limits and the roadmap
- Appendix A - verification results
- Appendix B - what changed in this pass
- Appendix C - command cheat-sheet

1. Where things stand

Cloudesk is a Next.js 16 application with PostgreSQL (Drizzle ORM) holding every piece of state: sessions, uploads, settings, jobs. One container image runs three roles - web, worker and migrate - so web servers are stateless and scale horizontally. 97 SQL migrations define the schema. A production build and the full test suite were run on a clean machine-like copy of the code for this guide (Appendix A).

1.1 Verified in this pass

Check	Result
TypeScript (next typegen + tsc --noEmit)	Pass (0 errors)
ESLint	Pass (0 problems)
Test suite (vitest, against Postgres)	Pass - 176 files, 1,341 tests, 0 failures (3 min 3 s)
Production build (next build)	Pass - 295 routes compiled (next build, production)
Production migration script (node scripts/migrate.mjs)	Pass - 97 migrations applied to the test database

1.2 Fixed in this pass (code)

- Docker image: a failed build no longer produces an image (the old '|| true' hid build errors); the build cache is not shipped; the image carries Amazon's RDS certificate bundle; a liveness health check replaces the database-dependent one.
- Migrations inside the image: scripts/migrate.mjs is plain JavaScript, so it runs in the production image (the old TypeScript script needed dev-only tools that the image removes). Compose, Kubernetes and the ECS migrate task use it.
- Database connection: pool size (DB_POOL_MAX), statement timeout and TLS are configurable; DATABASE_SSL_CA verifies RDS's certificate.
- Multi-server correctness: AI-task recovery no longer runs on every web server start (it failed tasks other servers were still running); the worker now fails only tasks untouched for 20 minutes, at start and every 5 minutes.
- Graceful shutdown: on SIGTERM the worker stops claiming jobs so a deploy never cuts a job in half.
- Health: /api/v1/health/live (liveness, no database) next to /api/v1/health (readiness, checks the database).
- Start-up checks: the encryption key is validated exactly as the cipher needs it (32 bytes, base64); warnings when there is no TLS to the database, no TRUST_PROXY, no email provider, or a worker without the single-replica acknowledgement.
- Image hygiene: .dockerignore excludes the video project, media, docs, tests and screenshots (hundreds of MB); Node 22 is pinned (.nvmrc, package.json engines).
- AWS artefacts: ECS task definitions for web, worker and migrate; a Caddy edge for customer domains; GitHub Actions for CI and for releases to ECS.

1.3 Must do before going live (not code - decisions and operations)

1. Put the code under version control and push it. The repository on this machine holds one commit (the Create Next App template) and the entire product is uncommitted. Nothing in section 8 works without a pushed repository.
2. Move development off iCloud Drive. iCloud evicts files from node_modules to free disk space; three separate check runs failed on this machine because evicted files loaded as empty modules. Builds must happen in CI, and local development should live in a folder iCloud does not sync (or be set to 'Keep downloaded').
3. Choose and verify an email provider (section 5). Without one, nobody can sign up: production sign-up sends a code by email.
4. Decide the region (ap-south-1, Mumbai, for Indian customers and DPDP data residency) and set DATA_REGION to match - it is shown to customers.
5. Run the restore drill (section 10) before taking customers.

2. Target architecture on AWS

A scalable, boring shape: managed Postgres, stateless web tasks behind an application load balancer, one worker task, and a small Caddy edge for customer domains. Everything is private except the two load balancers.

```
Internet
  |
  +-- Route 53 (yourdomain.com)
  |     app.yourdomain.com ---> ALB (ACM cert, HTTPS, AWS WAF)
  |                               |
  |                               +--> ECS Fargate "web" (APP_ROLE=web, 2..20 tasks, auto-scaled)
  |                                         |                ^
  |     sites.yourdomain.com --> NLB (TCP) --> ECS "edge" (Caddy, on-demand TLS, EFS for certs)
  |     (customer domains CNAME here)       |
  |                               ECS "worker" (exactly 1 task: schedulers + job queue)
  |                               ECS task "migrate" (one-off, every release)
  |                                         |
  |                               RDS PostgreSQL 16 (Multi-AZ, encrypted, PITR, private)
  |
  +-- Secrets Manager (DATABASE_URL, APP_ENCRYPTION_KEY, mail/payment/AI keys)
  +-- ECR (the image)    CloudWatch (logs, metrics, alarms)    S3 (later: media)
```

Component	Service	Why
Network	VPC, 2-3 AZs, public + private subnets, NAT gateway	Tasks and the database stay private; only load balancers are public.
App servers	ECS on Fargate (no servers to patch)	Stateless; scale by task count. 1 vCPU / 2 GB per web task to start.
Background work	ECS Fargate, 1 task	Mail sync, reminders, billing ticks, the job queue. Several schedulers are not safe to run twice.
Database	RDS PostgreSQL 16, Multi-AZ, gp3, encrypted, 14-35 day backups + PITR	All state lives here, uploads included (bytea). Needs pg_trgm (available on RDS).
Edge for the app	ALB + ACM + AWS WAF	TLS for app.yourdomain.com, managed rules, fleet-wide rate limits.
Edge for customer domains	NLB + Caddy (ECS) + EFS	Issues certificates on demand for domains customers connect; ALB/ACM cannot.
Secrets	Secrets Manager (or SSM Parameter Store)	Injected into tasks; never in the image or repository.
Images	ECR	Built by GitHub Actions, tagged by commit.
Logs / metrics	CloudWatch	JSON logs from stdout; alarms on 5xx, health, DB and task resources.
Email	Resend (simplest) or Amazon SES via SMTP	Sign-up codes, resets, notifications, invoices.

3. Build-out, step by step

Order matters: network, database, secrets, image, roles, cluster, services, DNS. Replace ACCOUNT_ID, REGION and yourdomain.com throughout. All commands assume the AWS CLI v2 is configured with an administrator profile.

3.1 Prerequisites

- An AWS account with billing alerts; a domain in Route 53 (or delegated to it).
- The repository pushed to GitHub (section 1.3).
- AWS CLI v2, Docker, Node 22 on your machine (for the first manual release; CI does the rest).
- A region decision - this guide uses ap-south-1 (Mumbai).

3.2 Network

1. Create a VPC (10.0.0.0/16) with 2 public and 2 private subnets across two AZs; an internet gateway; one NAT gateway (two for HA) for the private subnets.
2. Security groups: sg-alb (inbound 80/443 from 0.0.0.0/0), sg-web (inbound 3000 from sg-alb and sg-edge only), sg-edge (inbound 80/443 from 0.0.0.0/0 via the NLB), sg-db (inbound 5432 from sg-web, sg-worker, sg-migrate only), sg-efs (2049 from sg-edge).
3. VPC endpoints (optional but cheaper and safer): ECR (api + dkr), S3, Secrets Manager, CloudWatch Logs.

3.3 Database - RDS PostgreSQL

1. Engine PostgreSQL 16, Multi-AZ, instance db.t4g.large to start (2 vCPU / 8 GB), gp3 100 GB with autoscaling to 1 TB, storage encrypted (KMS), automated backups 14 days (35 for regulated customers), PITR on, deletion protection on, private subnets only, sg-db.
2. Parameter group: max_connections >= (web tasks x DB_POOL_MAX) + (2 x web tasks, LISTEN clients) + worker (DB_POOL_MAX + 2) + 20 headroom. For 6 web tasks at DB_POOL_MAX=10: 6x10 + 12 + 12 + 20 = 104 -> set 200. Keep rds.force_ssl=1.
3. Create the application database and a least-privilege user: a login role that owns the schema (Drizzle creates tables and the pg_trgm extension; on RDS 16 the master user can create pg_trgm, or run CREATE EXTENSION pg_trgm once as master).
4. Do not put PgBouncer (transaction mode) or RDS Proxy in front: chat and documents use LISTEN/NOTIFY, which needs real sessions. Size connections as above instead.
5. Build DATABASE_URL: postgres://cloudesk:PASSWORD@cloudesk-prod.xxxx.ap-south-1.rds.amazonaws.com:5432/cloudesk - without sslmode in the URL; TLS is turned on with DATABASE_SSL_CA (section 4).

```
psql "postgres://master:...@cloudesk-prod.xxxx.ap-south-1.rds.amazonaws.com:5432/postgres" <<SQL
create role cloudesk login password 'STRONG-PASSWORD';
create database cloudesk owner cloudesk;
\c cloudesk
create extension if not exists pg_trgm;
SQL
```

3.4 Secrets

Create one secret per value under the prefix cloudesk/prod/ (plain string secrets, not JSON) so each can be referenced from a task definition. The full list is in section 4.

```
aws secretsmanager create-secret --name cloudesk/prod/DATABASE_URL --secret-string 'postgres://cloudesk:...
aws secretsmanager create-secret --name cloudesk/prod/APP_ENCRYPTION_KEY --secret-string "$(openssl rand -b
aws secretsmanager create-secret --name cloudesk/prod/MAIL_FROM --secret-string 'Cloudesk <hello@yourdomain
aws secretsmanager create-secret --name cloudesk/prod/RESEND_API_KEY --secret-string 're_...'
# plus RAZORPAY_* / STRIPE_* / ANTHROPIC_API_KEY as you enable them
```

> APP_ENCRYPTION_KEY encrypts every stored credential (mailboxes, payment keys, integrations). Back it up in a second place (a sealed password-manager entry). If it is lost, those credentials are unrecoverable; it cannot be rotated in place.

3.5 Container image - ECR

```
aws ecr create-repository --repository-name cloudesk --image-scanning-configuration scanOnPush=true
aws ecr get-login-password | docker login --username AWS --password-stdin ACCOUNT_ID.dkr.ecr.REGION.amazona
docker build -f deploy/Dockerfile -t ACCOUNT_ID.dkr.ecr.REGION.amazonaws.com/cloudesk:v1.0.0 .
docker push ACCOUNT_ID.dkr.ecr.REGION.amazonaws.com/cloudesk:v1.0.0
```

Build on linux/amd64 (the task definitions request X86_64). On an Apple Silicon Mac add --platform linux/amd64, or let GitHub Actions build (section 8).

3.6 IAM roles

- cloudeskEcsExecutionRole: the AWS managed AmazonECSTaskExecutionRolePolicy plus secretsmanager:GetSecretValue on arn:aws:secretsmanager:REGION:ACCOUNT_ID:secret:cloudesk/prod/* and logs:CreateLogGroup.
- cloudeskEcsTaskRole: what the running app may call. Today nothing - the app talks only to RDS and the internet. Add S3 permissions here when media moves to S3.
- A GitHub OIDC role for releases (section 8): ECR push, ecs:RegisterTaskDefinition, ecs:UpdateService, ecs:RunTask, ecs:Describe*, iam:PassRole on the two roles above.

3.7 ECS cluster, task definitions and services

1. Create the cluster: aws ecs create-cluster --cluster-name cloudesk --capacity-providers FARGATE FARGATE_SPOT --settings name=containerInsights,value=enabled
2. Edit deploy/aws/task-web.json, task-worker.json and task-migrate.json: replace ACCOUNT_ID, REGION, the image tag, APP_URL, DATA_REGION and CUSTOM_DOMAIN_CNAME; drop secrets you do not use yet. Register them: aws ecs register-task-definition --cli-input-json file://deploy/aws/task-web.json (and the other two).
3. Create the ALB target group (section 3.8) first, then the web service: desired 2, min 2, max 20, private subnets, sg-web, assignPublicIp DISABLED, deployment circuit breaker with rollback, minimum healthy 100% / maximum 200%, health check grace 90 s. Register it with the target group on port 3000.
4. Auto-scaling for web: target tracking on ECSServiceAverageCPUUtilization 60% and ALBRequestCountPerTarget ~800 req/min per task; scale-in cooldown 300 s.

5. Worker service: desired 1, min 1, max 1, deployment minimum healthy 0% / maximum 100% so a rolling deploy never runs two workers at once. No load balancer.
6. Migrate: not a service. Run it as a one-off task before each release (section 7 and the deploy workflow).

```
aws ecs create-service --cluster cloudesk --service-name web --task-definition cloudesk-web \
  --desired-count 2 --launch-type FARGATE --health-check-grace-period-seconds 90 \
  --deployment-configuration "minimumHealthyPercent=100,maximumPercent=200,deploymentCircuitBreaker={enable
  --network-configuration "awsvpcConfiguration={subnets=[subnet-priv-a,subnet-priv-b],securityGroups=[sg-we
  --load-balancers "targetGroupArn=arn:aws:elasticloadbalancing:...:targetgroup/cloudesk-web/...,containerN

aws ecs create-service --cluster cloudesk --service-name worker --task-definition cloudesk-worker \
  --desired-count 1 --launch-type FARGATE \
  --deployment-configuration "minimumHealthyPercent=0,maximumPercent=100" \
  --network-configuration "awsvpcConfiguration={subnets=[subnet-priv-a,subnet-priv-b],securityGroups=[sg-wo
```

3.8 Load balancer, certificate, DNS, WAF

1. ACM: request a certificate for app.yourdomain.com (DNS validation in Route 53). Add sites.yourdomain.com only if you terminate customer sites on the ALB too (you will not - see section 6).
2. ALB in the public subnets with sg-alb. Listener 443 (ACM certificate, TLS policy ELBSecurityPolicy-TLS13-1-2-2021-06) -> target group cloudesk-web (IP targets, port 3000, health check path /api/v1/health, interval 15 s, healthy 2 / unhealthy 3, deregistration delay 30 s). Listener 80 -> redirect to 443. Idle timeout 120 s (long AI responses).
3. Route 53: A/AAAA alias app.yourdomain.com -> the ALB.
4. AWS WAF on the ALB: AWSManagedRulesCommonRuleSet, AWSManagedRulesKnownBadInputsRuleSet, AWSManagedRulesAmazonIpReputationList, plus a rate-based rule (for example 2000 requests / 5 minutes per IP). The app's own firewall runs behind this, per server.
5. Optional CloudFront in front of the ALB for static assets (_next/static/*) and DDoS absorption; set APP_URL to the CloudFront-fronted host and keep TRUST_PROXY=xff-last.

4. Environment and secrets

The app refuses to start in production without the first three. Everything marked 'secret' belongs in Secrets Manager and is referenced from the task definition's secrets block; the rest are plain environment entries.

Keys for email, billing, GitHub and AI can also be saved from Admin > Integrations; they are encrypted with APP_ENCRYPTION_KEY and reach every server within 30 seconds. An environment variable of the same name always wins.

Variable	Required	Value / notes
DATABASE_URL (secret)	yes	postgres://user:pass@host:5432/db - no sslmode; TLS via DATABASE_SSL_CA.
APP_ENCRYPTION_KEY (secret)	yes	openssl rand -base64 32 (exactly 32 bytes, base64). Never changes once data exists.
APP_URL	yes	https://app.yourdomain.com - cookies, callbacks and the CSRF origin check depend on it.
NODE_ENV	yes	production
APP_ROLE	web tasks	web on every web task; unset on the single worker.
TRUST_PROXY	yes on ALB	xff-last (ALB appends the client to X-Forwarded-For). Otherwise rate limits and bans can be spoofed.
DATABASE_SSL_CA	yes	/app/certs/rds-global-bundle.pem (shipped in the image).
DB_POOL_MAX	no	Connections per task, default 10. See section 3.3 for the sum.
MAIL_FROM (secret) + provider key	yes	Resend: RESEND_API_KEY; or POSTMARK_SERVER_TOKEN, SENDGRID_API_KEY, SMTP_URL (SES).
RAZORPAY_KEY_ID/KEY_SECRET/WEBHOOK_SECRET (secret)	to charge	Or STRIPE_SECRET_KEY/STRIPE_WEBHOOK_SECRET. Without them, plans are not enforced.
ANTHROPIC_API_KEY (secret)	for AI	Copilot, notes AI, mail AI. Without it only the deterministic planner runs. AI_MONTHLY_BUDGET_USD caps spend.
CUSTOM_DOMAIN_CNAME	customer domains	sites.yourdomain.com - the NLB/Caddy edge (section 6).
DATA_REGION	yes (shown to customers)	e.g. India - Mumbai (ap-south-1)
WORKER_SINGLETON_ACK	worker	1 - confirms you run exactly one worker.
AI_TASK_STALE_MIN	no	Minutes before an untouched AI task is failed by the worker (default 20).
GITHUB_CLIENT_ID/SECRET, GOOGLE_OAUTH_*, MICROSOFT_OAUTH_*	optional	Sign in with GitHub; Gmail / Microsoft 365 mailbox OAuth.
VOYAGE_API_KEY	optional	Semantic search for the site chatbot.

Variable	Required	Value / notes
EXEC_DOCKER	**leave unset**	**Python/Node sandboxes need a Docker daemon - not available on Fargate.**

5. Email, payments and AI

5.1 Email (required)

- Resend (recommended, fastest): verify yourdomain.com (SPF + DKIM records in Route 53), create an API key, set MAIL_FROM and RESEND_API_KEY. Add a DMARC record (v=DMARC1; p=quarantine; rua=mailto:dmarc@yourdomain.com).
- Amazon SES instead: verify the domain (Easy DKIM), move the account out of the SES sandbox, create SMTP credentials and set SMTP_URL=smtps://SMTP_USER:SMTP_PASS@email-smtp.ap-south-1.amazonaws.com:465 with SMTP_REQUIRE_TLS=1. Set up a bounce/complaint SNS topic.
- Test from Admin > Integrations > Email > Send a test, then sign up a real account on the production host before inviting anyone.

5.2 Payments

- Razorpay (India): create Plans for each paid tier, set RAZORPAY_KEY_ID, RAZORPAY_KEY_SECRET; add the webhook https://app.yourdomain.com/api/v1/billing/webhooks/razorpay with the subscription.* events and set RAZORPAY_WEBHOOK_SECRET.
- Stripe (international): two recurring Prices; webhook https://app.yourdomain.com/api/v1/billing/webhooks/stripe with checkout.session.completed, customer.subscription.*, invoice.paid, invoice.payment_failed.
- Customer stores have their own payment webhooks under /api/v1/store/{projectId}/webhooks/{provider}; they must stay reachable through the ALB and WAF (they are signed and bypass the CSRF wrapper by design).
- The platform never trusts a browser redirect as proof of payment; webhooks are verified server-side. Keep the webhook secrets in Secrets Manager.

5.3 AI

- Set ANTHROPIC_API_KEY and AI_MONTHLY_BUDGET_USD. AI tasks run on the web task that started them (cancel works only there); long tasks are capped by the worker's stale sweep.
- Monaco (the code editor) loads from jsDelivr and the CSP still allows unsafe-eval for it; self-hosting it is on the roadmap (section 14).

6. Customer domains and TLS

Customers connect their own domains to published sites. Certificates must be issued at run-time for hostnames you do not know in advance; ALB + ACM cannot do that, so a small Caddy edge does, approved by the app (GET /api/v1/domains/ask answers 200 only for a verified domain with a live deployment).

1. Build and push deploy/aws/caddy (FROM caddy:2-alpine) to ECR as cloudesk-edge.
2. Create an EFS file system (encrypted) in the private subnets with sg-efs; mount it at /data in the edge task definition so certificates and the ACME account survive restarts.
3. ECS service 'edge': 2 tasks, Fargate, public subnets with public IPs OR private subnets behind an NLB; environment APP_INTERNAL_HOST = the web service's Service Connect / Cloud Map name (enable ECS Service Connect on the web service, namespace cloudesk.internal, port name web:3000).
4. NLB (TCP 80 and 443 pass-through, no TLS termination) -> target groups for the edge tasks on 80 and 443. Health check TCP 443.
5. Route 53: sites.yourdomain.com A alias -> the NLB. Set CUSTOM_DOMAIN_CNAME=sites.yourdomain.com on the app; customers CNAME their domain to it (the app shows them this).
6. Caddy overwrites X-Forwarded-For; the app behind it would be correct with xff-first, but keep TRUST_PROXY=xff-last globally: Caddy's own address then becomes 'last', which is wrong. Better: have Caddy append (remove the header_up line) and keep xff-last everywhere. Choose one convention and document it.

> Keep the platform host (app.yourdomain.com) on the ALB with ACM. Only customer sites go through the edge. Rate-limit /api/v1/domains/ask at the WAF as well (it is already limited in the app).

7. First release and bootstrap

1. Run migrations as a one-off task: aws ecs run-task --cluster cloudesk --launch-type FARGATE --task-definition cloudesk-migrate --network-configuration "awsvpcConfiguration={subnets=[...],securityGroups=[sg-migrate],assignPublicIp=DISABLED}" and wait for exit code 0 (CloudWatch /cloudesk/migrate).
2. Start the web and worker services (section 3.7). Confirm https://app.yourdomain.com/api/v1/health returns {"status":"ok"} and the ALB targets are healthy.
3. Read the start-up log of one web task and one worker task in CloudWatch: any production_config warning must be understood (section 4).
4. Sign up the first account on the production host (email code must arrive). Then make it the platform owner from a bastion / psql session: update users set is_platform_admin = true where email = 'you@yourdomain.com';
5. Open /admin > Go-live and work through its checklist. Turn on two-step sign-in for yourself, then require it for admins (Admin > Settings > Admin security).
6. Admin > Integrations: confirm email, payments and AI show as set and pass their live tests.
7. Set Admin > Security > Firewall > trust proxy to match TRUST_PROXY (xff-last) so the console agrees with the environment.

8. CI/CD

Two workflows were added under .github/workflows/.

- ci.yml - on every push and pull request: npm ci, typecheck, lint, migrations into a Postgres service container, the full vitest suite, a production build, and npm audit (high+, non-blocking). Add the repository secret CI_APP_ENCRYPTION_KEY (any openssl rand -base64 32).
- deploy-aws.yml - on a v* tag or manually: builds the image, pushes it to ECR tagged with the commit, runs the migrate task and waits for exit 0, then registers new task definitions and rolls the web service, then the worker. It authenticates with GitHub OIDC (no long-lived AWS keys).

Repository configuration

Name	Type	Value
AWS_ROLE_ARN	secret	The OIDC role from section 3.6
AWS_REGION	variable	ap-south-1
ECR_REPOSITORY	variable	cloudesk
ECS_CLUSTER / ECS_SERVICE_WEB / ECS_SERVICE_WORKER	variables	cloudesk / web / worker
ECS_TASKDEF_MIGRATE	variable	cloudesk-migrate
ECS_SUBNETS / ECS_SECURITY_GROUP	variables	subnet-priv-a,subnet-priv-b / sg-migrate

Release discipline

- Every migration must be compatible with the version still running: add first, remove in a later release.
- Tag releases (git tag v1.0.1 && git push --tags); roll back by re-deploying the previous tag (migrations are forward-only, so design them that way).
- Branch protection on main: CI must pass; at least one review.

9. Monitoring, logs and alarms

- Logs: the app writes JSON lines to stdout; the awslogs driver ships them to /cloudesk/web, /cloudesk/worker, /cloudesk/migrate. Set retention (90 days) and, for regulated customers, an export to S3.
- Metric filters on the web log group: level=error count; status>=500 count; 'production_config' warnings at start.
- Alarms (SNS -> email/Slack): ALB HTTPCode_Target_5XX_Count > 20 in 5 min; UnHealthyHostCount >= 1 for 2 periods; TargetResponseTime p95 > 2 s; ECS CPU/Memory > 80% (web and worker); worker RunningTaskCount < 1; RDS CPU > 75%, FreeStorageSpace < 20%, DatabaseConnections > 80% of max_connections, FreeableMemory low, ReplicaLag if a replica exists; Secrets Manager/ECR deploy failures (EventBridge on deployment circuit-breaker rollbacks).
- In-app: Admin > Health (database latency, connections, migrations, job backlog, failing deployments, domains, email, sign-ins) and Admin > Scalability (request metrics, growth projections).
- Health endpoints: ALB -> /api/v1/health (readiness, checks the database); container -> /api/v1/health/live (liveness).
- Gaps to close soon: no error-tracking service (add Sentry or an OpenTelemetry exporter to CloudWatch/X-Ray) and no request-id header yet.

10. Backups and disaster recovery

- RDS automated backups 14-35 days with point-in-time recovery; Multi-AZ for failover. Enable cross-region automated backup replication (or AWS Backup with a copy to a second region) for regional disaster recovery.
- Uploads and attachments live in Postgres today, so the database backup is the only backup; size storage and IOPS accordingly (section 11).
- Restore drill before launch and quarterly: restore the latest snapshot to a scratch instance, run the migrate task against it with a throw-away image, log in, open a workspace, download an upload. Record the time it took - that is your RTO.
- Secrets: Secrets Manager keeps versions; additionally keep APP_ENCRYPTION_KEY sealed offline (section 3.4).
- Infrastructure: keep the task definitions, service settings and WAF rules in the repository (deploy/aws); moving them to Terraform or CDK is the next step.

11. Scaling

11.1 Web tier

- Stateless: scale by task count. Start with 2 tasks of 1 vCPU / 2 GB; auto-scale on CPU 60% and ALB request count. Fargate Spot for part of the fleet is fine for web (never for the worker).
- Published customer sites, the API and the app share the same tasks; if sites get heavy traffic, add CloudFront in front and raise the max task count first.
- Session and rate-limit state is in Postgres (shared); the firewall's per-IP counters are per task (AWS WAF covers the fleet).

11.2 Worker

- Exactly one task. It polls the job queue (SKIP LOCKED) and runs every scheduler. If it is ever CPU-bound, give it more vCPU before considering a second one - several schedulers would double-send.
- A second worker becomes possible once those schedulers claim rows atomically (section 14).

11.3 Database

- Connections: max_connections >= web tasks x (DB_POOL_MAX + 2) + worker x (DB_POOL_MAX + 2) + 20. Lower DB_POOL_MAX to 6-8 if you run many small tasks.
- Instance: db.t4g.large -> db.m7g.large / xlarge as CPU passes 60% sustained. Storage gp3 with autoscaling; raise provisioned IOPS if uploads grow (they are bytea rows).
- Read replica: not used by the app today; add one for BI/exports only when needed.
- Housekeeping is built in: the worker prunes rate limits, firewall events, logs past retention and old exports.

11.4 Media to S3 (next step)

All uploads are stored in Postgres. That is fine to a few tens of GB. Before that, add the S3 adapter the schema already anticipates (src/db/schema.ts notes it): store object keys in the assets table, serve through CloudFront with signed URLs, and migrate existing rows with a one-off job. Give the task role s3:PutObject/GetObject on the bucket.

11.5 Rough monthly cost (ap-south-1, on-demand, excluding data transfer)

Item	Size	USD / month (approx.)
ECS Fargate web	2 x (1 vCPU, 2 GB), scaling to 6	60 - 180
ECS Fargate worker	1 x (1 vCPU, 2 GB)	30
ECS Fargate edge (Caddy)	2 x (0.25 vCPU, 0.5 GB)	15
RDS PostgreSQL	db.t4g.large Multi-AZ, 100 GB gp3, backups	190 - 230
ALB + NLB + WAF	2 load balancers, WAF managed rules	60 - 80

Item	Size	USD / month (approx.)
NAT gateway	**1 (2 for HA)**	**35 - 70**
CloudWatch, ECR, Secrets Manager, EFS		20 - 40

Order of magnitude 400-650 USD/month for a production-grade, highly available setup; Savings Plans and Reserved Instances cut 30-40% once usage is steady.

12. Security hardening

- TRUST_PROXY=xff-last behind the ALB (otherwise clients can spoof their address past rate limits, bans and the admin IP allowlist). The app now warns at start if it is unset.
- Security groups as in 3.2; the database accepts connections only from app tasks; no public IPs on web/worker tasks.
- TLS everywhere: ACM on the ALB (TLS 1.2+), rds.force_ssl=1 with the shipped CA bundle, HSTS is sent for two years (add 'preload' and submit to hstspreload.org once you are sure).
- Secrets only in Secrets Manager; rotate database and provider credentials on a schedule (APP_ENCRYPTION_KEY cannot be rotated in place - keep it sealed and backed up).
- AWS WAF managed rules + rate rule on the ALB; the in-app firewall (Admin > Security) adds bans, strikes and an admin IP allowlist per server.
- Admin accounts: two-step sign-in required (Admin > Settings > Admin security); use the admin IP allowlist for /admin.
- CSP: the platform UI still allows 'unsafe-inline' and 'unsafe-eval' for Monaco from jsDelivr; self-host Monaco and drop unsafe-eval (roadmap). Published customer sites already get a stricter, script-free policy.
- Dependencies: ECR scan-on-push is enabled; CI runs npm audit. Review high findings weekly.
- Data residency: ap-south-1 keeps data in India; set DATA_REGION accordingly and document it in your privacy notice (DPDP).
- Sandboxes: in-app JavaScript runs in QuickJS inside the web process (3 s / 64 MB cap); leave EXEC_DOCKER unset on Fargate.
- Maintenance mode (Admin > Notices) makes workspaces read-only with a message while published sites keep serving - use it for risky migrations.

13. Go-live checklist

1. Code committed and pushed; CI green on main; a v1.0.0 tag.
2. VPC, security groups, RDS Multi-AZ with PITR, deletion protection on.
3. All secrets in Secrets Manager; APP_ENCRYPTION_KEY backed up offline.
4. Image in ECR; migrate task exited 0; web (>=2 tasks) and worker (1 task) services stable.
5. ALB healthy targets; https://app.yourdomain.com loads; /api/v1/health is ok; HTTP redirects to HTTPS.
6. Start-up logs show no production_config warnings you have not accepted.
7. Email: domain verified (SPF, DKIM, DMARC); test email received; a real sign-up completed with the emailed code.
8. Payments: webhooks registered with secrets; a test subscription paid and reflected (test mode first, then live keys).
9. First platform admin set; two-step sign-in on; Admin > Go-live checklist complete.
10. Customer-domain edge live: sites.yourdomain.com resolves; a test domain CNAMEd, verified and served with a valid certificate.
11. WAF attached; TRUST_PROXY=xff-last; firewall trust proxy set to match.
12. CloudWatch alarms created and routed to a channel somebody watches; log retention set.
13. Restore drill done and timed; backup replication to a second region enabled.
14. Load test against staging (for example k6 at 2-3x expected peak) with auto-scaling observed.
15. Legal: privacy notice with DATA_REGION, terms, DPDP consent flows reviewed; support email answered by a human.
16. Runbook: who is on call, how to roll back (re-deploy previous tag), how to enable maintenance mode, where logs and alarms live.

14. Known limits and the roadmap

Limit (verified in the code)	Impact	Plan
Uploads/attachments stored in Postgres (bytea)	Database size, IOPS and backup size grow with media	S3 adapter + CloudFront signed URLs (schema already anticipates it)
Several worker schedulers are not claim-safe	Exactly one worker task	Add atomic row claims (education reminders, chat reminders, mail follow-ups, helpdesk auto-close, growth tick), then allow 2+ workers
AI tasks run on the web task that started them	Cancel works only there; a task dies if its task is replaced mid-run (worker fails it after 20 min)	Move AI tasks onto the job queue
Firewall per-IP limits are per server	Fleet-wide limits rely on AWS WAF	Move counters to Postgres (as API rate limits already are) or ElastiCache
QuickJS sandbox runs inside the web process	A run can block the event loop up to 3 s	worker_threads or a separate service
Python/Node exec needs Docker	Unavailable on Fargate	A dedicated EC2/Firecracker runner, or keep it off
No error-tracking / tracing / request ids	Slower diagnosis	Sentry or OpenTelemetry -> CloudWatch/X-Ray; a request-id header in the route wrapper
CSP allows unsafe-inline/unsafe-eval; Monaco from jsDelivr	Weaker XSS defence on the platform UI	Self-host Monaco, nonce-based CSP
No infrastructure-as-code yet	Manual console/CLI set-up	Terraform or CDK from the task definitions in deploy/aws
Mail was tested against a real provider only once (Hostinger); AI only with stubs	Verify on your providers before launch	Live tests from Admin > Integrations; a staging environment

Appendix A - verification results

All checks were run on 29 October 2026 on a clean copy of the working tree outside iCloud Drive (rsync of the source, npm ci, then the same steps CI runs). The copy was needed because iCloud had evicted 22,000+ files under node_modules, which made three in-place runs fail inside node_modules before reaching the application code.

Step	Outcome	Detail
npm ci	ok	Node 22; lockfile install from scratch
npm run typecheck	pass	next typegen + tsc --noEmit: 0 errors
npm run lint	pass	eslint: 0 problems
npm run build	pass	next build (production): 295 routes, proxy (middleware) compiled
node scripts/migrate.mjs --test	pass	The new production migration script applied all 97 migrations to the test database
npx vitest run	pass	176 files, 1,341 tests, 0 failures, 182 s (unit + integration against Postgres)

Failures and how they were handled

- First full run: 1,340 of 1,341 passed. The one failure was tests/integration/platform-production.test.ts expecting the old start-up message ('at least 32 characters') after the encryption-key check was tightened to exactly 32 base64 bytes. The test was updated to assert the stricter check and the new warnings; the whole suite was then re-run and passed completely.
- An earlier run of the copy failed typecheck and build with 'Cannot find module ../media/assets' - a mistake in the copy itself (an unanchored rsync exclude dropped src/server/media). Re-synced correctly, both passed. Not a code problem.

Appendix B - what changed in this pass

File	Change
deploy/Dockerfile	Build failures fail the image; dev deps pruned; .next/cache dropped; RDS CA bundle added; liveness health check; STOPSIGNAL
scripts/migrate.mjs (new)	Plain-JS migrations for the production image; TLS like the app; .env.local only when present
src/db/index.ts	DB_POOL_MAX, DB_STATEMENT_TIMEOUT_MS, DATABASE_SSL_CA / DATABASE_SSL=no-verify, idle timeout
src/server/ai/tasks.ts	recoverInterruptedTasks(staleMs): fails only tasks untouched for staleMs
src/instrumentation.ts	Recovery on the worker only (start + every 5 min, AI_TASK_STALE_MIN); SIGTERM stops the job worker
src/server/jobs/queue.ts	stopWorker()
src/app/api/v1/health/live/route.ts (new)	Liveness endpoint without a database check
src/server/platform/boot.ts	Exact encryption-key check; warnings for DB TLS, TRUST_PROXY, email provider, worker singleton
.dockerignore, .nvmrc, package.json	Smaller build context; Node 22; db:migrate:prod script
deploy/docker-compose.yml, deploy/kubernetes.yaml, docs/DEPLOY.md	node scripts/migrate.mjs; liveness probe; AWS notes; pooler and proxy guidance
deploy/aws/* (new)	ECS task definitions (web, worker, migrate); Caddy edge Dockerfile + Caddyfile
.github/workflows/ci.yml, deploy-aws.yml (new)	CI on every push; release to ECS on tags

Appendix C - command cheat-sheet

```
# Local development
cp .env.example .env.local && npm ci && npm run db:migrate && npm run dev

# All checks (what CI runs)
npm run typecheck && npm run lint && npm run db:migrate:test && npx vitest run && npm run build

# Image
docker build --platform linux/amd64 -f deploy/Dockerfile -t cloudesk .
docker run --rm --env-file .env.production cloudesk node scripts/migrate.mjs      # migrate
docker run --rm --env-file .env.production -e APP_ROLE=web -p 3000:3000 cloudesk  # a web server

# Release (after `git tag v1.0.1 && git push --tags`, or manually)
aws ecs run-task --cluster cloudesk --task-definition cloudesk-migrate ...        # migrate
aws ecs update-service --cluster cloudesk --service web --force-new-deployment    # roll web
aws ecs update-service --cluster cloudesk --service worker --force-new-deployment # then worker

# Roll back: deploy the previous image tag with the same two commands.
# Logs:   aws logs tail /cloudesk/web --follow
# Health: curl -s https://app.yourdomain.com/api/v1/health
```